



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема роботи

Використання патерна проєктування **Adapter (Адаптер)** у програмному забезпеченні.

Мета роботи

Ознайомитися з патерном проєктування **Adapter**, його призначенням та структурою, а також реалізувати даний патерн у програмному проєкті для забезпечення сумісності між несумісними інтерфейсами без зміни існуючого коду.

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	3
Завдання:	3
Діаграма класів:	3
Опис реалізації:	3
Вихідний код	7
Контрольні запитання.....	9
Висновок	12

Теоретичні відомості

Патерн проєктування Adapter

Adapter (Адаптер) – це структурний патерн проєктування, який дозволяє об'єктам з несумісними інтерфейсами працювати разом. Патерн діє як «перехідник», перетворюючи інтерфейс одного класу на інтерфейс, який очікує клієнт.

Патерн Adapter застосовується у випадках, коли:

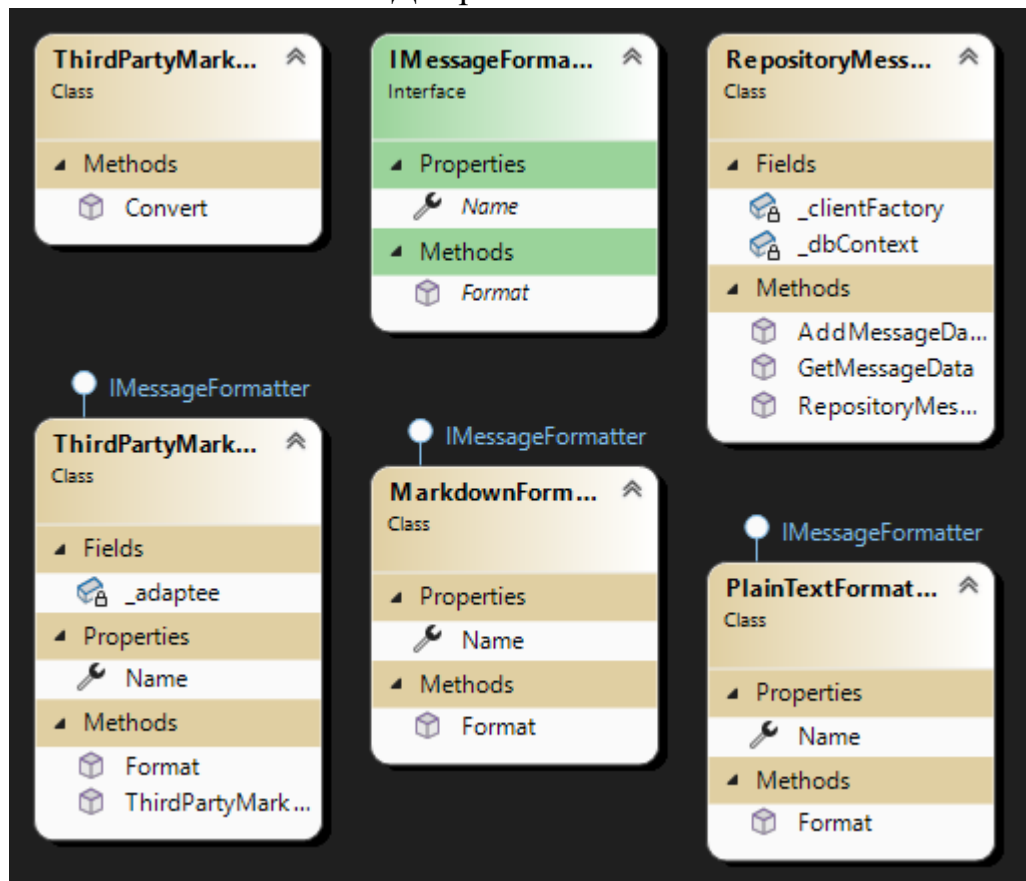
- необхідно використати сторонній або застарілий код;
- інтерфейс наявного класу не відповідає вимогам системи;
- зміна стороннього коду є неможливою або небажаною.

Хід виконання:

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів:



Реалізовані класи та інтерфейс для цієї роботи

Опис реалізації:

Опис реалізації в лабораторній роботі

У даній лабораторній роботі патерн Adapter було реалізовано для задачі інтеграції стороннього (гіпотетичного) формatera Markdown, API якого не відповідає внутрішньому інтерфейсу форматування повідомлень у системі.

Цільовий інтерфейс (Target)

У системі використовується інтерфейс:

`IMessageFormatter`

який визначає стандартний спосіб форматування повідомлень методом `Format(Message)` та використовується репозиторієм повідомлень.

Адаптований клас (Adaptee)

Було змодельовано сторонній або застарілий клас:

`ThirdPartyMarkdownFormatter`

який має несумісний інтерфейс:

- інший метод (`Convert`);
- інші параметри (рядок тексту, ідентифікатор користувача, час).

Даний клас не може бути використаний напряду без зміни існуючого коду.

Адаптер (Adapter)

Для узгодження інтерфейсів було створено клас:

`ThirdPartyMarkdownAdapter`

який:

- реалізує інтерфейс `IMessageFormatter`;
- всередині делегує виклик методу сторонньому класу `ThirdPartyMarkdownFormatter`;
- трансліює виклик `Format(Message)` у відповідний виклик `Convert(...)`.

Таким чином, адаптер перетворює несумісний інтерфейс стороннього класу у формат, очікуваний системою.

Використання адаптера

Клас RepositoryMessages не було змінено з точки зору логіки роботи з форматерами. Він, як і раніше, працює з колекцією IMessageFormatter, отриманою через Dependency Injection, та обирає реалізацію під час виконання програми.

Це означає, що адаптер є прозорим для клієнтського коду.

Відповідність патерну Adapter

Реалізація відповідає патерну Adapter з таких причин:

1. Наявність несумісних інтерфейсів – сторонній формater не відповідає IMessageFormatter.
2. Цільовий інтерфейс (Target) – IMessageFormatter.
3. Адаптований клас (Adaptee) – ThirdPartyMarkdownFormatter.
4. Адаптер (Adapter) – ThirdPartyMarkdownAdapter, який узгоджує інтерфейси.
5. Незмінність клієнта – RepositoryMessages не знає про існування стороннього API.

Приклад використання

Отримання повідомлень із застосуванням різних реалізацій форматування:

```
var messages = await repositoryMessages.GetMessageData(chatId, "md");
```

```
var messages2 = await repositoryMessages.GetMessageData(chatId, "plain");
```

```
var messages3 = await repositoryMessages.GetMessageData(chatId, "thirdparty-md");
```

У третьому випадку використовується адаптер, який делегує форматування сторонньому класу.

Переваги реалізації

- забезпечено сумісність зі стороннім API без його модифікації;
- зменшено зв'язаність між репозиторієм і конкретними реалізаціями форматування;
- спрощено розширення системи (додавання нових адаптерів);

- реалізація відповідає принципу Open/Closed.

Вихідний код

```
// Formatting/ThirdPartyMarkdownAdapter.cs
```

```
using Skype.Data;
```

```
namespace Skype.Formatting
```

```
{
```

```
    // Adapter: exposes IMessageFormatter and delegates to ThirdPartyMarkdownFormatter.
```

```
    public class ThirdPartyMarkdownAdapter : IMessageFormatter
```

```
    {
```

```
        public string Name => "thirdparty-md";
```

```
        private readonly Formatting.Legacy.ThirdPartyMarkdownFormatter _adaptee;
```

```
        // Adaptee injected via DI to keep things testable
```

```
        public ThirdPartyMarkdownAdapter(Formatting.Legacy.ThirdPartyMarkdownFormatter adaptee)
```

```
        {
```

```
            _adaptee = adaptee;
```

```
        }
```

```
        public string Format(Message message)
```

```
        {
```

```
            // Adapt IMessageFormatter.Format(Message) to the adaptee's Convert(...)
```

```
            return _adaptee.Convert(message.Text, message.OwnerId, message.TimeSend);
```

```
        }
```

```
    }
```

```
}
```

```
// Formatting/ThirdPartyMarkdownFormatter.cs
```

```
namespace Skype.Formatting.Legacy
```

```
{
```

```
    // Simulates a third-party / legacy formatter with a different API.
```

```
    public class ThirdPartyMarkdownFormatter
```

```
{  
    public string Convert(string text, int ownerId, DateTime time)  
    {  
        // Simulated conversion (escape and decorate) – replace with real library  
call.  
        var escaped = System.Net.WebUtility.HtmlEncode(text);  
        return $"**User:{ownerId}** _{time:yyyy-MM-dd HH:mm}_\n\n{escaped}";  
    }  
}  
}
```

Контрольні запитання

1. Яке призначення шаблону «Адаптер»?

Шаблон **Adapter** (Адаптер) призначений для узгодження інтерфейсів несумісних класів і надання можливості їх спільної роботи без зміни існуючого коду.

2. Структура шаблону «Адаптер»

Client

|

v

Target (interface)

^

|

Adapter -----> Adaptee

3. Класи шаблону «Адаптер» та їх взаємодія

- **Target** – інтерфейс, з яким працює клієнт.
- **Adaptee** – існуючий або сторонній клас з несумісним інтерфейсом.
- **Adapter** – реалізує Target і делегує виклики Adaptee.
- **Client** – використовує Target, не знаючи про Adaptee.

4. Різниця між адаптером на рівні об'єктів та класів

- **Об'єктний адаптер** використовує композицію (Adapter містить Adaptee).
- **Класовий адаптер** використовує наслідування (Adapter наслідує Adaptee).

Об'єктний адаптер є гнучкішим і використовується частіше.

5. Яке призначення шаблону «Будівельник»?

Шаблон **Builder (Будівельник)** відокремлює процес побудови складного об'єкта від його представлення, дозволяючи створювати різні варіанти об'єкта з одного процесу конструювання.

6. Структура шаблону «Будівельник»

Director

|

v

Builder (interface)

^

|

ConcreteBuilder ----> Product

7. Класи шаблону «Будівельник» та їх взаємодія

- **Builder** – інтерфейс побудови об'єкта.
- **ConcreteBuilder** – конкретна реалізація побудови.
- **Product** – створюваний об'єкт.
- **Director** – керує послідовністю кроків побудови.

8. Коли варто застосовувати шаблон «Будівельник»?

Шаблон Builder доцільно застосовувати, коли:

- об'єкт має багато параметрів або складну структуру;
- необхідно створювати різні варіанти одного об'єкта;
- потрібно ізолювати клієнта від деталей створення об'єкта.

9. Яке призначення шаблону «Команда»?

Шаблон **Command (Команда)** інкапсулює запит у вигляді об'єкта, дозволяючи параметризувати клієнтів різними запитами та підтримувати відкладене виконання або скасування операцій.

10. Структура шаблону «Команда»

Client

|

v

Command (interface)

^

|

ConcreteCommand ----> Receiver

^

|

Invoker

11. Класи шаблону «Команда» та їх взаємодія

- **Command** – інтерфейс команди.
- **ConcreteCommand** – реалізує команду та зберігає посилання на Receiver.
- **Receiver** – виконує реальні дії.
- **Invoker** – викликає команду.
- **Client** – створює команди та налаштовує Invoker.

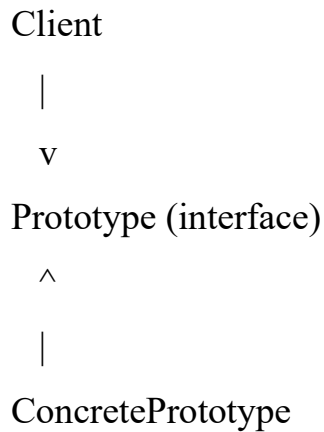
12. Як працює шаблон «Команда»?

Клієнт створює об'єкт команди, який інкапсулює виклик методу Receiver. Invoker викликає метод Execute() команди, не знаючи деталей виконання. Таким чином досягається слабка зв'язаність між викликом і виконанням дії.

13. Яке призначення шаблону «Прототип»?

Шаблон **Prototype (Прототип)** дозволяє створювати нові об'єкти шляхом клонування існуючих екземплярів без прив'язки до їх конкретних класів.

14. Структура шаблону «Прототип»



15. Класи шаблону «Прототип» та їх взаємодія

- **Prototype** – інтерфейс клонування.
- **ConcretePrototype** – реалізує механізм копіювання.
- **Client** – створює нові об’єкти шляхом клонування прототипу.

16. Приклади використання шаблону «Ланцюжок відповідальності»

Шаблон **Chain of Responsibility** застосовується у випадках:

- обробки подій у GUI (ланцюжок обробників);
- логування з різними рівнями (debug → info → error);
- фільтрації HTTP-запитів (middleware);
- перевірок доступу та валідації даних.

Висновок

У ході виконання лабораторної роботи було реалізовано патерн проєктування **Adapter**, який дозволив інтегрувати сторонній (гіпотетичний) форматер у систему без зміни існуючого клієнтського коду. Застосування патерна забезпечило гнучкість, розширюваність та чистоту архітектури програмного забезпечення.